

Contents

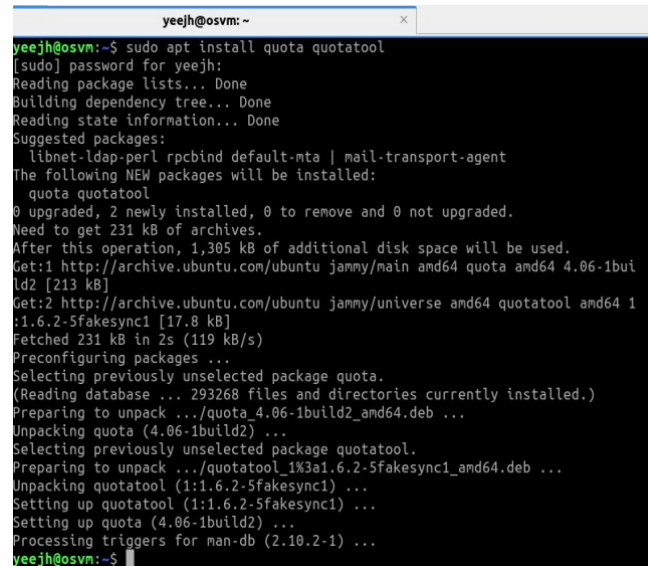
Part A	3
Task 1: Configure Storage Quota	3
Task 2: Logical Volume Management (LVM) – Configure Logical Volume	13
Task 3: RAID Configuration	18
Task 4: Synchronisation with semaphores	27

Part A

Task 1: Configure Storage Quota

Install required tools: Launch **QTerminal** and Install 'quota' and 'quotatool' packages using the following command,

```
sudo apt install quota quotatool
```

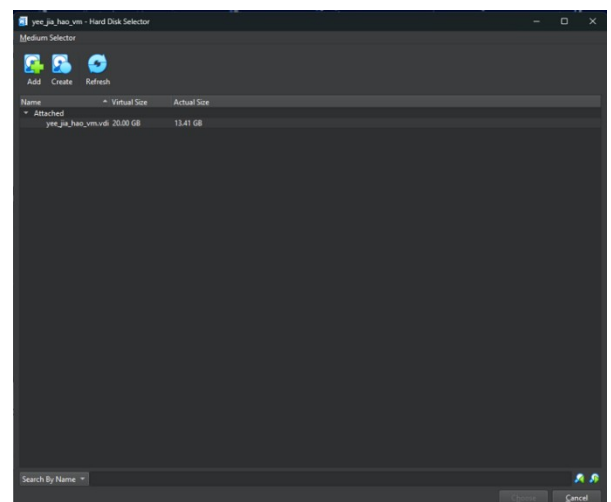
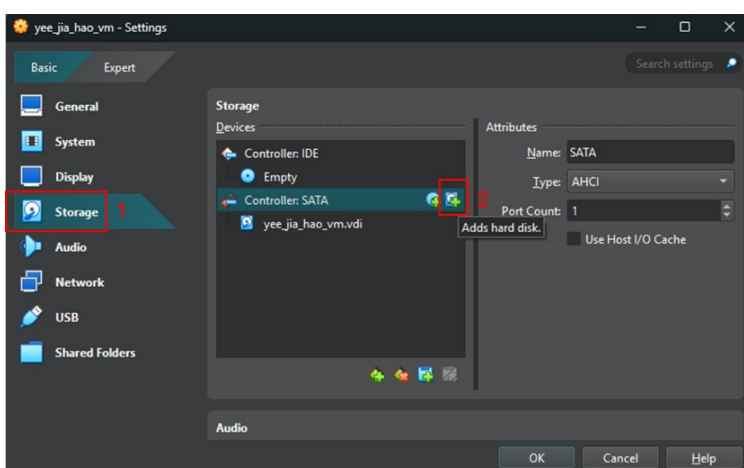


```
yeejh@osvm: ~  
yeejh@osvm:~$ sudo apt install quota quotatool  
[sudo] password for yeejh:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Suggested packages:  
  libnet-ldap-perl rpcbind default-mta | mail-transport-agent  
The following NEW packages will be installed:  
  quota quotatool  
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.  
Need to get 231 kB of archives.  
After this operation, 1,305 kB of additional disk space will be used.  
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 quota amd64 4.06-1build2 [213 kB]  
Get:2 http://archive.ubuntu.com/ubuntu jammy/universe amd64 quotatool amd64 1:1.6.2-5fakesync1 [17.8 kB]  
Fetched 231 kB in 2s (119 kB/s)  
Preconfiguring packages ...  
Selecting previously unselected package quota.  
(Reading database ... 293268 files and directories currently installed.)  
Preparing to unpack .../quota_4.06-1build2_amd64.deb ...  
Unpacking quota (4.06-1build2) ...  
Selecting previously unselected package quotatool.  
Preparing to unpack .../quotatool_1%3a1.6.2-5fakesync1_amd64.deb ...  
Unpacking quotatool (1:1.6.2-5fakesync1) ...  
Setting up quotatool (1:1.6.2-5fakesync1) ...  
Setting up quota (4.06-1build2) ...  
Processing triggers for man-db (2.10.2-1) ...  
yeejh@osvm:~$
```

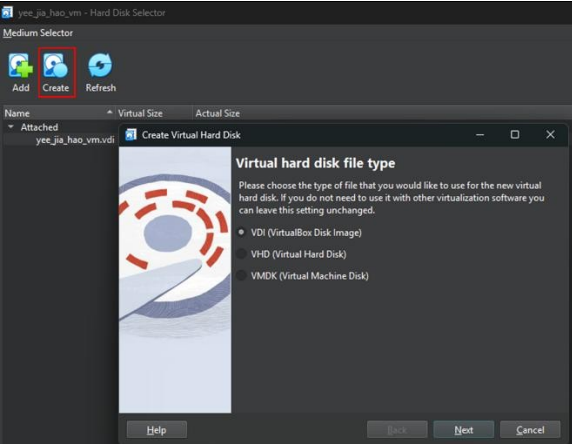
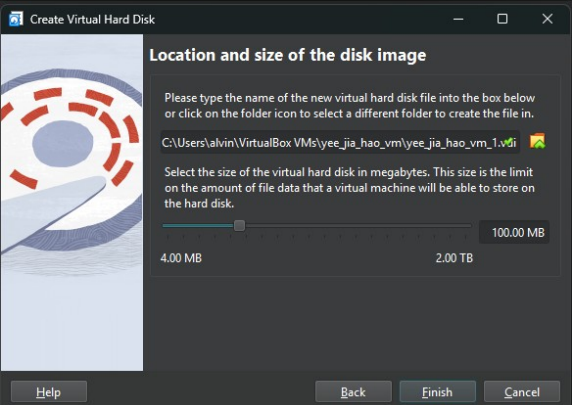
1. List the steps to create partition and file system in the additional disk. Include the relevant command(s) used in each step.

Step 1: Add 100.00MB Hard Disk to Lubuntu VM

- a. Make sure the VM is shut down, then go to VM settings and navigate to **Storage** section and add a new Hard Disk to **Controller: SATA** device. The **Hard Disk Selector** window on the right side will pop up.



- b. Click on **Create** and “Create Virtual Hard Disk” window will pop up, then follow the following steps and set up,

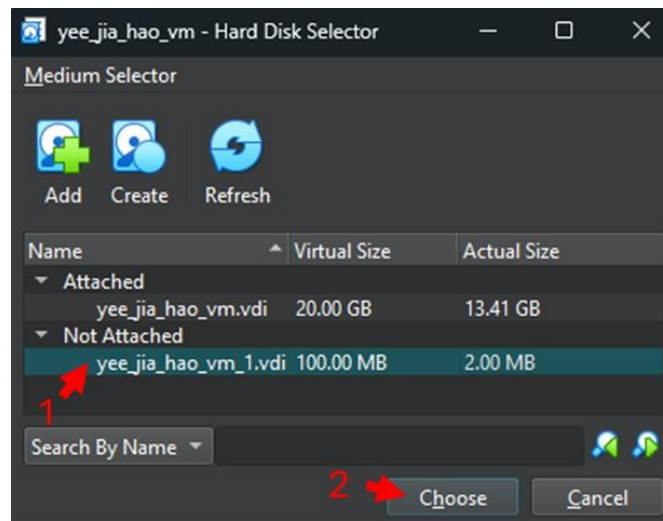
Step	Installation Screen	Set Up
1		- Select VDI (VirtualBox Disk Image) - Click Next
2		Click Next
3		- Set the size of virtual hard disk to 100.00MB - Click Finish

- c. The “Hard Disk Selector” should now show the new 100.00MB Hard Disk under “Not Attached” dropdown.

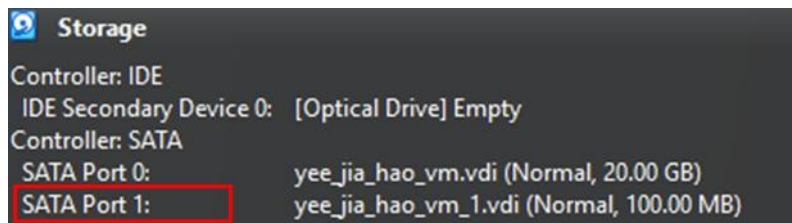
Name	Virtual Size	Actual Size
▼ Attached		
yee_jia_hao_vm.vdi	20.00 GB	13.41 GB
▼ Not Attached		
yee_jia_hao_vm_1.vdi	100.00 MB	2.00 MB

** The Actual Size represents the initial allocated size of Hard Disk, while the Virtual Size determines the maximum size of the Hard Disk to be allocated.*

- d. Choose the new Hard Disk and apply it to the Lubuntu VM.



- e. After adding, the Lubuntu's VM settings will show the new hard disk,



- f. Boot the VM again, and check if the 100MB new hard disk is added to the machine by using the command below,

```
lsblk --exclude 7 -o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT,SIZE
```

Command/ Prefixes	Comment
lsblk	Used to list all the block devices including like Hard Disks and Partitions.
--exclude 7	Remove loop devices.
--o NAME, FSTYPE, LABEL, UUID, FSAVAIL, FSUSE%, MOUNTPOINT, SIZE	Specify the needed information column to display: NAME – Disk or Partition Name. FSTYPE – File System Type. LABEL – The label assigned to the File System (FS). UUID – The unique identifier for the FS. FSUSE% – The percentage of space used. MOUNTPOINT – Location where the device is mounted. SIZE – The size of device.

```
yeejh@osvm:~$ lsblk --exclude 7 -o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT,SIZE
NAME FSTYPE LABEL UUID FSAVAIL FSUSE% MOUNTPOINT SIZE
sda
└─sda1 ext4 bc10a32a-7047-48ac-b614-0eef60de2e6 9.8G 44% / 20G
sdb 100M
sr0 1024M
yeejh@osvm:~$
```

Step 2: Create a Partition on the New 100.00MB hard disk

- a. From the Device Listing from 'e', it is determined that the new 100.00MB Hard Disk has the name 'sdb'.
- b. Create a GPT partition table for disk 'sdb',

```
sudo parted /dev/sdb mklabel GPT
```

```
yeejh@osvm:~$ sudo parted /dev/sdb mklabel GPT
[sudo] password for yeejh:
Information: You may need to update /etc/fstab.
```

* The message "Information: You may need to update /etc/fstab." indicate successful partition table creation.

- c. Create a primary partition using the whole disk space,

```
sudo parted -a optimal /dev/sdb mkpart primary ext4 2048s 100%
```

* Start after 1st sector (2048s OR 1MiB) to ensure the partition is aligned with the disk's physical sectors. This alignment is important to maintain the disk performance.

* 1MiB is consider wasted in this case as a trade-off for disk performance. However, the 1MiB space can be used to store Hard Disk metadata like for MBR and GPT partition table.

- d. Check the created partition and take note on the new partition name, **sdb1**.

```
yeejh@osvm:~$ lsblk --exclude 7 -o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT,SIZE
NAME    FSTYPE LABEL  UUID                                  FSAVAIL FSUSE% MOUNTPOINT  SIZE
sda
└─sda1  ext4              bc10a32a-7047-48ac-b614-0eef60de2e6  9.3G   47% /          20G
sdb
└─sdb1
sr0                                           1024M
```

Step 3: Format the Partition as ext4 File System

- a. Format the created partition, **sdb1** into **ext4** File System using the following command,

```
sudo mkfs.ext4 /dev/sdb1
```

```
yeejh@osvm:~$ sudo mkfs.ext4 /dev/sdb1
mkfs 1.46.5 (30-Dec-2021)
Creating filesystem with 25339 4k blocks and 25344 inodes
Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done
```

- b. Finally, the partition 'sdb1' is created containing ext4 filesystem and it is uniquely identified by Ubuntu system by UUID. Therefore, the partitioned file system is ready to be used.

```
yeejh@osvm:~$ lsblk --exclude 7 -o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT,SIZE
NAME FSTYPE LABEL UUID FSAVAIL FSUSE% MOUNTPOINT SIZE
sda
└─sda1 ext4 bc10a32a-7047-48ac-b614-0eef60de2e6 9.4G 46% / 20G
sdb
└─sdb1 ext4 e972ff73-ba29-4639-8a2c-c339841d7c34 100M 99M
```

Step 4: Mount the Partition on Ubuntu VM

- a. Create a folder '**/mnt/data**' for mounting the partition,

```
sudo mkdir /mnt/data
```

- b. Edit the '**/etc/fstab**' file to add an entry for the **sdb1** partition. Use its UUID to mount it to the folder '**/mnt/data**'. Ensure the changes are applied for automatic mounting on boot.

```
sudo nano /etc/fstab
NAME, FSTYPE, LABEL, UUID, FSAVAIL, FSUSE%, MOUNTPOINT, SIZE
```

Add in the following and save the modification,

```
UUID=e972ff73-ba29-4639-8a2c-c339841d7c34 /mnt/data
ext4 defaults 0 2
```

* *UUID will differ from machine to machine, copy it from **Step 3 b** above.*

```
yeejh@osvm: ~
GNU nano 6.2 /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a device; this may
# be used with UUID= as a more robust way to name devices that works even if
# disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options> <dump> <pass>
UUID=bc10a32a-7047-48ac-b614-0eef60de2e6 / ext4 defaults 0 1
UUID=e972ff73-ba29-4639-8a2c-c339841d7c34 /mnt/data ext4 defaults 0 2
```

- c. Check the device and partition again, the partition **‘/dev/sdb1’** is now mounted to **‘/mnt/data’**,

```
yeejh@osvm:~$ lsblk --exclude 7 -o NAME,FSTYPE,LABEL,UUID,FSAVAIL,FSUSE%,MOUNTPOINT,SIZE
NAME    FSTYPE LABEL  UUID                                  FSAVAIL FSUSE% MOUNTPOINT  SIZE
sda
└─sda1  ext4      bc10a32a-7047-48ac-b614-0eef60de2e6  9.4G    46% /           20G
sdb
└─sdb1  ext4      e972ff73-ba29-4639-8a2c-c339841d7c34  81.8M   0% /mnt/data  99M
```

** Although the FS usage percentage is 0%, the FS available size is smaller than the allocated size. This is because ext4 FS reserved some space for system operation.*

2. List the steps to configure and apply the quota for **student823**. Include the relevant command(s) used in each step.

Step 1: Enable User Quota for the Created Partition

- a. Edit the `/etc/fstab` file again, replace “defaults” of the mounted partition to “usrquota,grpquota” of the 100.00MB partition and save it,

```
sudo nano /etc/fstab
```

```
NAME, FSTYPE, LABEL, UUID, FSAVAIL, FSUSE%, MOUNTPOINT, SIZE
```

```
# <file system>          <mount point> <type> <options> <dump> <pass>
UUID=bc10a32a-7047-48ac-b614-0eeff60de2e6 /          ext4    defaults  0 1
UUID=e972ff73-ba29-4639-8a2c-c339841d7c34 /mnt/data  ext4    defaults,usrquota 0 2
```

- b. Remount the `/mnt/data`

```
sudo mount -o remount /mnt/data
```

```
yeejh@osvm:~$ sudo mount -o remount /mnt/data
```

* No error message means success remount.

- c. Check Kernel support for user quota and create `/mnt/data/aquota.user`.

```
sudo quotacheck -cum /mnt/data
```

```
yeejh@osvm:~$ sudo quotacheck -cum /mnt/data
```

* `-cum` prefix is used to only update or create `'aquota.user'` and not `'aquota.group'`.

* No error message means success check and quota file creation/update.

- d. Activate the user quota,

```
sudo quotaon -u /mnt/data
```

```
yeejh@osvm:~$ sudo quotaon -u /mnt/data
```

* `-u` prefix is used to only activate user quota.

* No error message means success user quota activation.

Step 2: Create User

- a. Create user “student823”,

```
sudo adduser student823
```

```
yeejh@osvm:~$ sudo adduser student823
Adding user `student823' ...
Adding new group `student823' (1001) ...
Adding new user `student823' (1001) with group `student823' ...
Creating home directory `/home/student823' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for student823
Enter the new value, or press ENTER for the default
  Full Name []: student823
    Room Number []:
    Work Phone []:
    Home Phone []:
      Other []:
Is the information correct? [Y/n] y
yeejh@osvm:~$
```


Step 3: Apply Quota for student823

- a. Assign user quota to “student823”,

```
sudo edquota -u student823
```

Enter the following and save it,

```
/dev/sdb1 4 50M 60M 1 0 0
```

Bolded value: Use the default value and should not be changed.

```
yeejh@osvm: ~  
GNU nano 6.2 /tmp//EdP.avWEWte *  
Disk quotas for user student823 (uid 1001):  
Filesystem      blocks      soft      hard      inodes      soft      hard  
/dev/sdb1         4         50M        60M         1          0          0
```

Column (Left to Right)	Comment
Filesystem	- The name or identifier for the file system directory name that need to managed by quota. - Configuring student823 quota information for the ‘/dev/sdb1’ file system.
Blocks	- Current Block (Disk Space) used. Don’t change the value.
Soft	- A soft limit for the space usage in block. Where the set limit can be exceeded temporarily with grace period of 7 days by default. - Value ‘50M’ mean the soft limit is set to 51200 Blocks or 50MiB.
Hard	- A hard limit for the space usage in block. The true maximum limit and user cannot exceed it. - Value ‘60M’ mean the user can only use up to 61440 Blocks or 60MiB file system.
Inodes	- Current Inodes (Numbers of File or Directory) used. Don’t change the value.
Soft	Soft limit for inode, where 0 mean no limit is set.
Hard	Hard limit for inode, where 0 mean no limit is set.

Step 4: Create Directory for student823

- a. Create a directory '/mnt/data/student823'

```
sudo mkdir /mnt/data/student823
```

- b. Change ownership of the directory '/mnt/data/student823' to "student823",

```
sudo chown student823:student823 /mnt/data/student823
```

```
yeejh@osvm:~$ ll /mnt/data
total 36
drwxr-xr-x 4 root    root    4096 Apr  5 02:59 ./
drwxr-xr-x 3 root    root    4096 Apr  4 22:10 ../
-rw----- 1 root    root    7168 Apr  5 02:27 aquota.user
drwx----- 2 root    root    16384 Apr  4 21:52 lost+found/
drwxr-xr-x 2 student823 student823 4096 Apr  5 02:59 student823/
```

- c. Remove all the permissions (read, write, execute) granted to the group,

```
sudo chmod 700 /mnt/data/student823
```

```
drwx----- 2 student823 student823 4096 Apr 19 13:25 student823
```

Step 5: Create 55MiB and 65MiB Test Files for student823

- a. Login as student823 and create a 55MiB (~58MB) and 65MiB (~68MB) test files at '/home/student823',

```
dd if=/dev/zero of=/home/student823/File55MiB bs=1M count=55
dd if=/dev/zero of=/home/student823/File65MiB bs=1M count=65
```

```
student823@osvm: ~
student823@osvm:~$ dd if=/dev/zero of=/home/student823/File55MiB bs=1M count=55
55+0 records in
55+0 records out
57671680 bytes (58 MB, 55 MiB) copied, 0.157365 s, 366 MB/s
student823@osvm:~$ dd if=/dev/zero of=/home/student823/File65MiB bs=1M count=65
65+0 records in
65+0 records out
68157440 bytes (68 MB, 65 MiB) copied, 0.225696 s, 302 MB/s
student823@osvm:~$
```

Name	Type	Size	Modified	Created	Owner	Group
Public	folder		5/04/25 3:19 PG	5/04/25 3:19 PG	student823	student823
Templates	folder		5/04/25 3:19 PG	5/04/25 3:19 PG	student823	student823
Videos	folder		5/04/25 3:19 PG	5/04/25 3:19 PG	student823	student823
File55MiB	unknown	55.0 MiB	18/04/25 5:27 PTG	18/04/25 5:27 PTG	student823	student823
File65MiB	unknown	65.0 MiB	18/04/25 5:27 PTG	18/04/25 5:27 PTG	student823	student823

Free space: 9.0 GiB (Total: 19.5 GiB)

Step 5: Perform Quota Testing on student823

To review changes, the command below is used to view the **student823** quota information (Using “root”)

```
sudo repquota -a
/mnt/data/student823
```

```
yeejh@osvm:~$ sudo repquota -a
*** Report for user quotas on device /dev/sdb1
Block grace time: 7days; Inode grace time: 7days
Block limits          File limits
User      used  soft  hard  grace  used  soft  hard  grace
-----
root      --    20     0     0         2     0     0
student823 --    4 51200 61440         1     0     0
```

* The block grace time is 7 days by default. This mean if user had used the block exceeded the soft limit for 7 days. Once the hard limit is enforced, the user student823 will be restricted from writing any new data to the filesystem until their usage falls below the limit.

* Currently, student823 had only used 4 blocks, 1 inode and no grace period is imposed.

- i. Copy a file that is larger than 50 MB but less than 60 MB (File55MiB).

```
student823@osvm:~$ cp File55MiB /mnt/data/student823/
student823@osvm:~$ ls /mnt/data/student823/
File55MiB
student823@osvm:~$
```

```
yeejh@osvm:~$ sudo repquota -a
*** Report for user quotas on device /dev/sdb1
Block grace time: 7days; Inode grace time: 7days
Block limits          File limits
User      used  soft  hard  grace  used  soft  hard  grace
-----
root      --    20     0     0         2     0     0
student823 +- 56324 51200 61440 6days    2     0     0
```

* **Student823** has used 56,324 blocks (~55MiB), exceeded the soft block limit but remained below the hard limit. As a result, a 7-day grace period for block usage has been imposed, allowing **student823** to continue writing files up to ~5 MiB in the 7-day period.

- ii. Remove the file copied in step (i).

```
student823@osvm:~$ rm /mnt/data/student823/File55MiB
student823@osvm:~$ ls /mnt/data/student823/
student823@osvm:~$
```

- iii. Copy a file that is larger than 60 MB (File65MiB) and error should occur,

```
student823@osvm:~$ cp File65MiB /mnt/data/student823/
cp: error writing '/mnt/data/student823/File65MiB': Disk quota exceeded
```

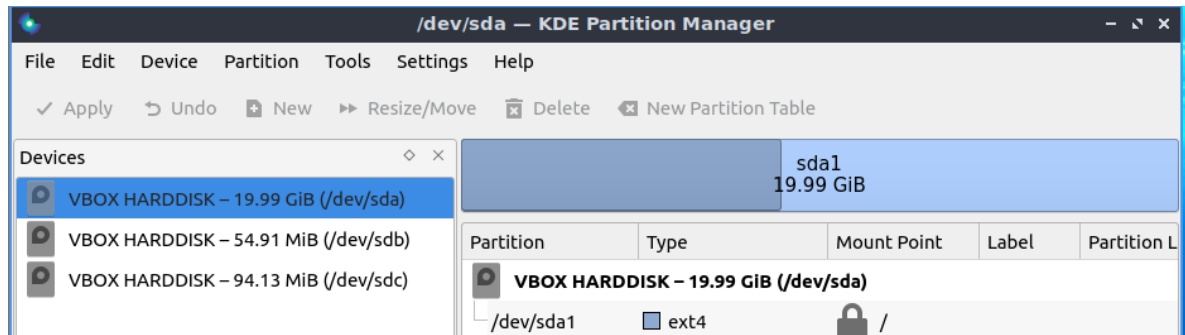
```
yeejh@osvm:~$ sudo repquota -a
*** Report for user quotas on device /dev/sdb1
Block grace time: 7days; Inode grace time: 7days
Block limits          File limits
User      used  soft  hard  grace  used  soft  hard  grace
-----
root      --    20     0     0         2     0     0
student823 +- 61440 51200 61440 6days    2     0     0
```

* Student823 has used up all 61440 blocks (60 MiB) and reached the hard limit, therefore, student823 can no longer write file to that directory and an error message “Disk quota exceeded” will be shown.

Task 2: Logical Volume Management (LVM) – Configure Logical Volume

Step 1: Add 2 hard disks with 60MB and 100MB

- 2 hard disks are detected in KDE Partition Manager



Steps 2: Create a volume group with using 60MB hard disk and named as vg-recovery

- To create a volume group with a name,

```
sudo vgcreate vg-recovery /dev/sdb
```

```
levin@osvm: ~  
levin@osvm:~$ sudo vgcreate vg-recovery /dev/sdb  
[sudo] password for levin:  
Physical volume "/dev/sdb" successfully created.  
Volume group "vg-recovery" successfully created  
levin@osvm:~$ sudo pvs  
PV          VG          Fmt  Attr  PSize  PFree  
/dev/sdb    vg-recovery  lvm2 a--  56.00m 56.00m  
levin@osvm:~$ sudo vgs  
VG          #PV #LV #SN Attr   VSize VFree  
vg-recovery  1   0   0 wz--n- 56.00m 56.00m
```

Step 3: Create a logical volume and name it as lv-user

- To create a logical volume with a name,

```
sudo lvcreate -l 100%FREE -n lv-user vg-recovery
```

```
levin@osvm: ~  
vg-recovery  1   0   0 wz--n- 56.00m 56.00m  
levin@osvm:~$ sudo lvcreate -l 100%FREE -n lv-user vg-recovery  
Logical volume "lv-user" created.  
levin@osvm:~$ sudo lvs  
LV          VG          Attr      LSize  Pool Origin Data%  Meta%  Move Log Cp  
y%Sync Convert  
lv-user     vg-recovery -wi-a----- 56.00m
```

Step 4: Create an ext4 filesystem on the logical volume, lv-user

- a. To create ext4 filesystem on the logical volume,

```
sudo mkfs.ext4 /dev/vg-recovery/lv-user
```

```
levin@osvm: ~  
lv-user vg-recovery -wi-a----- 56.00m  
levin@osvm:~$ sudo mkfs.ext4 /dev/vg-recovery/lv-user  
mke2fs 1.46.5 (30-Dec-2021)  
Creating filesystem with 14336 4k blocks and 14336 inodes  
  
Allocating group tables: done  
Writing inode tables: done  
Creating journal (1024 blocks): done  
Writing superblocks and filesystem accounting information: done
```

Step 5: Create a mount point directory, /mnt/lvm and mount to the logical volume

- a. To create directory

```
sudo mkdir -p /mnt/lvm
```

- b. To mount the logical volume to the mount point

```
sudo mount /dev/vg-recovery/lv-user /mnt/lvm
```

```
levin@osvm: ~  
Writing superblocks and filesystem accounting information: done  
levin@osvm:~$ sudo mkdir -p /mnt/lvm  
levin@osvm:~$ sudo mount /dev/vg-recovery/lv-user /mnt/lvm
```

```
levin@osvm: ~  
ext4 1.0 d752ee19-ad82-43cc-9a14-44660540e356 9.4G 47%  
/  
sdb LVM2_m LVM2 q9oqnf-YH9T-Pbxc-NW9p-c03k-d0hB-6stYlu  
└vg--recovery-lv--user  
  ext4 1.0 c8035ccc-f836-4379-9fd1-bb28f2bd90b8 44.5M 0%  
/mnt/lvm  
sdc  
  
sr0
```

Step 6: Copy a file that is larger than the logical volume size

- a. Error of writing files to logical volume occurs due to insufficient space. To fix this,

```
sudo cp /home/levin/Downloads/dummyfile.img
```

```
levin@osvm: ~  
cp: cannot stat 'home/levin/Downloads/dummyfile.img': No such file or directory  
levin@osvm:~$ sudo cp /home/levin/Downloads/dummyfile.img /mnt/lvm  
cp: error writing '/mnt/lvm/dummyfile.img': No space left on device  
levin@osvm:~$ sudo df -h /mnt/lvm  
Filesystem                Size      Used Avail Use% Mounted on  
/dev/mapper/vg--recovery-lv--user 49M    48M      0 100% /mnt/lvm
```

Step 7: Create a GPT partition table, then create a 50 MB primary partition with an ext4 filesystem and use the remaining space to create another primary partition

- a. Command used:

- `sudo parted /dev/sdc`
- `mklabel gpt`
- `mkpart primary ext4 1MB 51MB`
- `mkpart primary ext4 51MB 100%`

```
levin@osvm:~$ sudo parted /dev/sdc  
GNU Parted 3.4  
Using /dev/sdc  
Welcome to GNU Parted! Type 'help' to view a list of commands.  
(parted) mklabel gpt  
(parted) mkpart primary ext4 1MB 51MB  
(parted) mkpart primary ext4 51MB 100%  
(parted) print  
Model: ATA VBOX HARDDISK (scsi)  
Disk /dev/sdc: 105MB  
Sector size (logical/physical): 512B/512B  
Partition Table: gpt  
Disk Flags:  
  
Number  Start   End     Size    File system  Name      Flags  
1       1049kB  51.4MB  50.3MB  ext4         primary  
2       51.4MB  105MB   53.5MB  ext4         primary
```

Step 8: Create a physical volume

```
sudo pvcreate /dev/sdc1
```

```
levin@osvm:~$ sudo pvcreate /dev/sdc1  
Physical volume "/dev/sdc1" successfully created.
```

Step 9: Add the partition /dev/sdc1 to the volume group, vg-recovery

```
sudo vgextend vg-recovery /dev/sdc1
```

```
levin@osvm:~$ sudo vgextend vg-recovery /dev/sdc1
Volume group "vg-recovery" successfully extended
levin@osvm:~$ sudo vgs
VG          #PV #LV #SN Attr   VSize   VFree
vg-recovery  2   1   0 wz--n- 100.00m 44.00m
levin@osvm:~$ sudo pvs
PV          VG          Fmt  Attr PSize  PFree
/dev/sdb    vg-recovery  lvm2 a--  56.00m   0
/dev/sdc1   vg-recovery  lvm2 a--  44.00m 44.00m
```

Step 10: Extend the logical volume to include the new space

```
sudo lvextend -l +100%FREE /dev/vg-recovery/lv-user
```

```
levin@osvm:~$ sudo lvextend -l +100%FREE /dev/vg-recovery/lv-user
Size of logical volume vg-recovery/lv-user changed from 56.00 MiB (14 extents) to 100.00 MiB (25 extents).
Logical volume vg-recovery/lv-user successfully resized.
```

- The logical volume has expanded.

Before:

```
levin@osvm:~$ sudo lvs
LV          VG          Attr      LSize   Pool Origin Data%  Meta%  Move Log Cp
lv-user     vg-recovery  -wi-ao--- 56.00m
```

After:

```
levin@osvm:~$ sudo lvs
LV          VG          Attr      LSize   Pool Origin Data%  Meta%  Move Log C
lv-user     vg-recovery  -wi-ao--- 100.00m
```

Step 11: Resize the ext4 filesystem in the logical volume

```
sudo resize2fs /dev/vg-recovery/lv-user
```

```
levin@osvm:~$ sudo resize2fs /dev/vg-recovery/lv-user
resize2fs 1.46.5 (30-Dec-2021)
Filesystem at /dev/vg-recovery/lv-user is mounted on /mnt/lvm; online resizing required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/vg-recovery/lv-user is now 25600 (4k) blocks long.
```

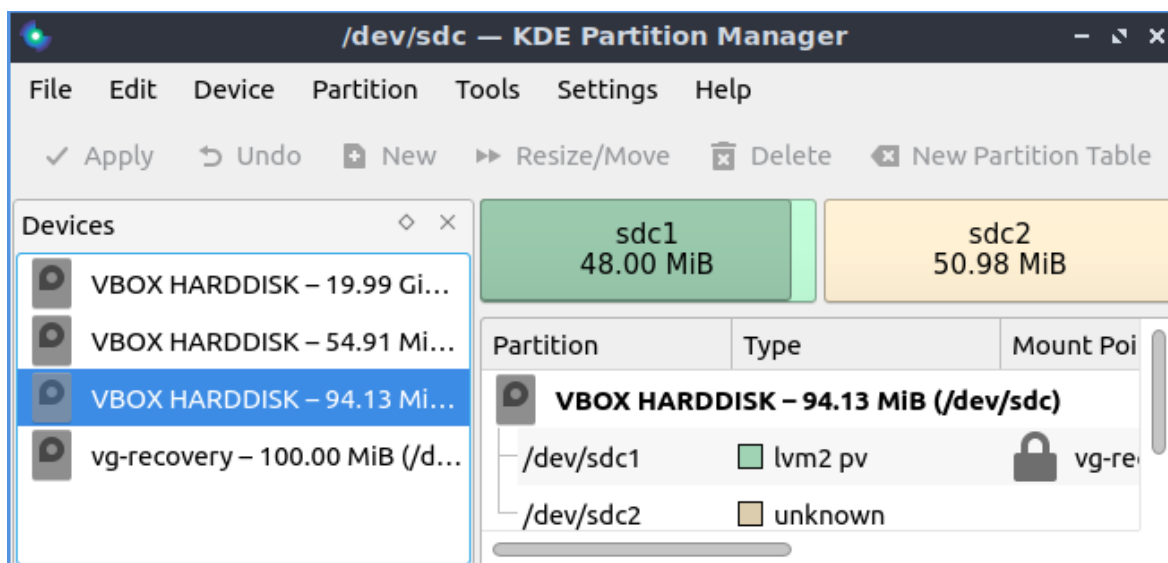
Step 12: Copy the file to /mnt/lvm again

- a. File copied successfully.

```
sudo cp /home/levin/Downloads/dummyfile.img /mnt/lvm
```

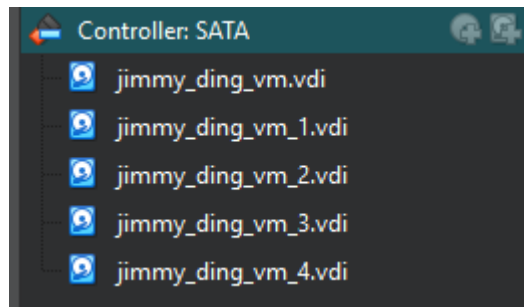
```
levin@osvm:~$ sudo cp /home/levin/Downloads/dummyfile.img /mnt/lvm
levin@osvm:~$ sudo ls -l /mnt/lvm
total 71696
-rw-r--r-- 1 root root 73400320 Apr  5 13:06 dummyfile.img
drwx----- 2 root root  16384 Apr  5 12:01 lost+found
levin@osvm:~$
```

- b. The summary of partitions created.

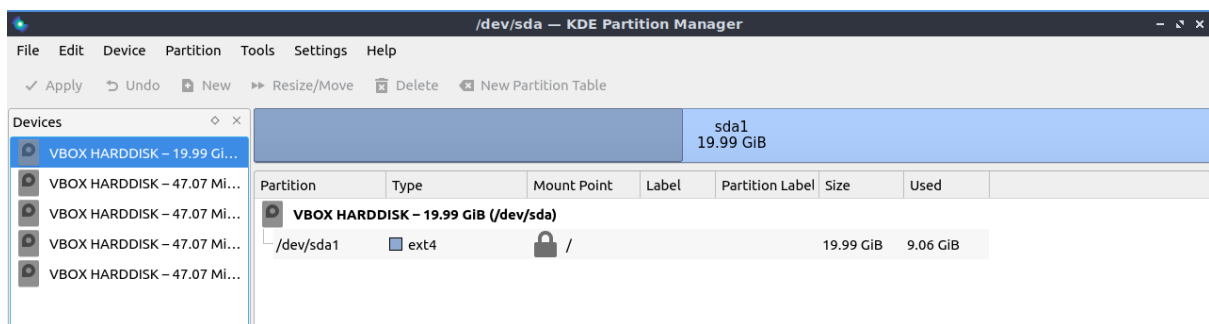


Task 3: RAID Configuration

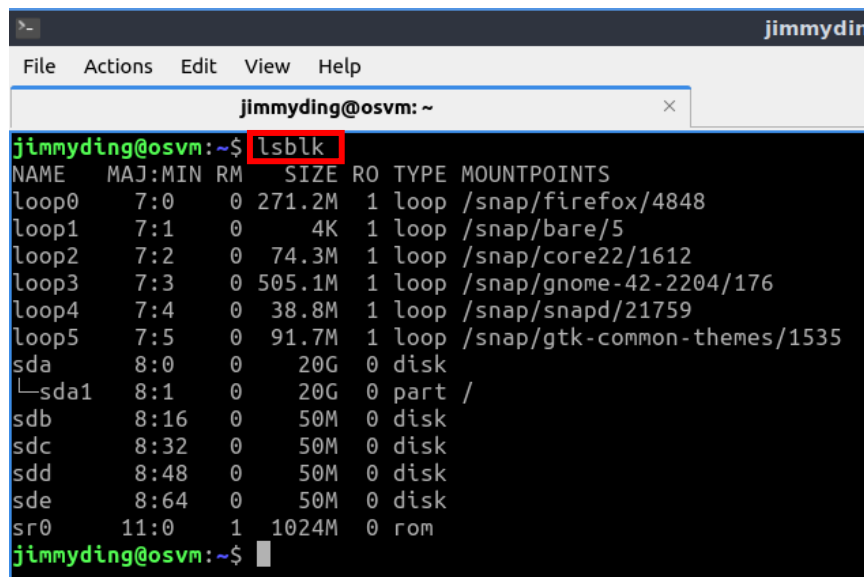
Step 1: Add 4 hard disks with 50MB each



New hard disks added



KDE Partition Manager



Viewing disks via “lsblk”. The new hard disks names are

- sdb
- sdc
- sdd
- sde

Step 2: Installing mdadm package

- a. The mdadm package is then installed with:

```
sudo apt update && sudo apt install mdadm -y
```

Step 3: Creating RAID level 5

- a. Before the RAID5 arrays are created, the involved disks are to be partitioned first. This is done with “fdisk”.

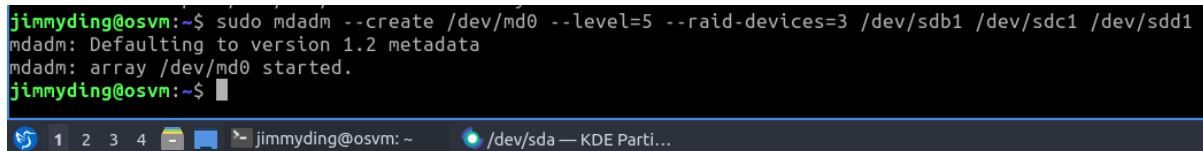
Command	Description
Command: n	Create new partition
Partition type: p	Primary partition
Partition number: 1	Make the partition the first partition
First Sector: 2048	Begin at sector number 2048 on the disk
Last Sector: 102399	Ends at sector number 102399 on the disk
Command: t	Change partition type
Hex code or alias: fd	This signals to the OS that this partition is meant to be part of a software RAID array
Command: w	Write the new changes to the disk and exit.

- b. This is repeated for the remaining 2 disks.

```
jimmyding@osvm: ~  
File Actions Edit View Help  
jimmyding@osvm: ~  
Password:  
su: Authentication failure  
jimmyding@osvm:~$ sudo fdisk /dev/sdb  
  
Welcome to fdisk (util-linux 2.37.2).  
Changes will remain in memory only, until you decide to write them.  
Be careful before using the write command.  
  
Device does not contain a recognized partition table.  
Created a new DOS disklabel with disk identifier 0xdb0f5cc6.  
  
Command (m for help): n  
Partition type  
  p   primary (0 primary, 0 extended, 4 free)  
  e   extended (container for logical partitions)  
Select (default p): p  
Partition number (1-4, default 1): 1  
First sector (2048-102399, default 2048):  
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-102399, default 102399):  
  
Created a new partition 1 of type 'Linux' and of size 49 MiB.  
  
Command (m for help): t  
Selected partition 1  
Hex code or alias (type L to list all): fd  
Changed type of partition 'Linux' to 'Linux raid autodetect'.  
  
Command (m for help): w  
The partition table has been altered.  
Calling ioctl() to re-read partition table.  
Syncing disks.  
  
jimmyding@osvm:~$
```

- b. Once the disks have been partitioned, a RAID level 5 named “/dev/md0” can be created. This is done by specifying the raid level and the disks that we want to add to the array.

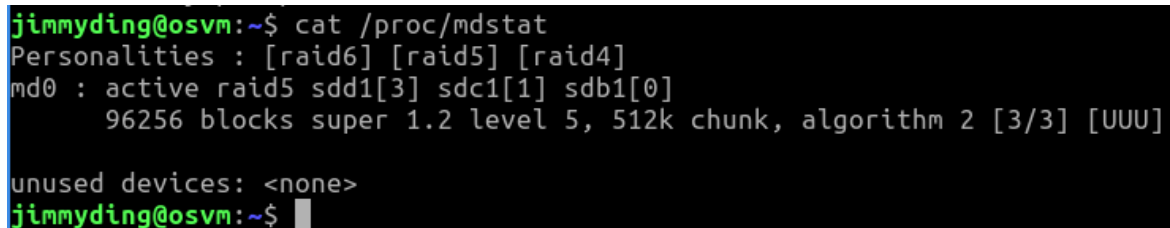
```
mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb1  
/dev/sdc1 /dev/sdd1
```

A terminal window showing the execution of the mdadm command to create a RAID 5 array. The user is jimmyding@osvm. The command is 'sudo mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb1 /dev/sdc1 /dev/sdd1'. The output shows 'mdadm: Defaulting to version 1.2 metadata' and 'mdadm: array /dev/md0 started.'

```
jimmyding@osvm:~$ sudo mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb1 /dev/sdc1 /dev/sdd1  
mdadm: Defaulting to version 1.2 metadata  
mdadm: array /dev/md0 started.  
jimmyding@osvm:~$
```

- c. To show the real-time status of the active software RAID array on the system, the command is used,

```
cat /proc/mdstat
```

A terminal window showing the output of the 'cat /proc/mdstat' command. It displays the RAID configuration for md0, which is an active raid5 array using three disks: sdd1[3], sdc1[1], and sdb1[0]. It also shows the number of blocks (96256), the RAID level (5), the chunk size (512k), and the algorithm (2).

```
jimmyding@osvm:~$ cat /proc/mdstat  
Personalities : [raid6] [raid5] [raid4]  
md0 : active raid5 sdd1[3] sdc1[1] sdb1[0]  
      96256 blocks super 1.2 level 5, 512k chunk, algorithm 2 [3/3] [UUU]  
  
unused devices: <none>  
jimmyding@osvm:~$
```

Step 4: Displaying Information

- a. The information of “dev/md0” is shown with the command:

```
mdadm --detail /dev/md0
```

```
jimmyding@osvm:~$ sudo mdadm --detail /dev/md0
/dev/md0:
    Version : 1.2
  Creation Time : Fri Apr  4 11:33:46 2025
    Raid Level : raid5
    Array Size : 96256 (94.00 MiB 98.57 MB)
  Used Dev Size : 48128 (47.00 MiB 49.28 MB)
    Raid Devices : 3
    Total Devices : 3
  Persistence : Superblock is persistent

    Update Time : Fri Apr  4 11:33:47 2025
      State : clean
  Active Devices : 3
 Working Devices : 3
 Failed Devices : 0
  Spare Devices : 0

    Layout : left-symmetric
    Chunk Size : 512K

Consistency Policy : resync

        Name : osvm:0 (local to host osvm)
        UUID : 0c7ad10e:637cb255:768be582:50671067
        Events : 18

   Number   Major   Minor   RaidDevice State
     0         8       17         0   active sync  /dev/sdb1
     1         8       33         1   active sync  /dev/sdc1
     3         8       49         2   active sync  /dev/sdd1
jimmyding@osvm:~$
```

Step 5: Creating ext4 filesystem on /dev/md0

- a. Command used to create ext4 file system

```
mkfs.ext4 /dev/md0
```

```
jimmyding@osvm:~$ sudo mkfs.ext4 /dev/md0
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 24064 4k blocks and 24064 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done

jimmyding@osvm:~$
```

Step 6: Mounting /dev/md0 to /mnt/rd5

- a. By default, the “/mnt” directory in lubuntu 22.04.5 is empty. So, before the mounting, “rd5” folder has to be created first in the “/mnt” directory.

```
mkdir /mnt/rd5
```

```
jimmyding@osvm:~$ ls /mnt
rd5
jimmyding@osvm:~$
```

- b. Once the folder has been created, the “/dev/md0” is mounted to the folder.

```
mount /dev/md0 /mnt/rd5
```

- c. To test the mounted device, a text file has been created and copied into the folder.

```
jimmyding@osvm:~$ cat /home/jimmyding/Documents/testing
Hello World!
jimmyding@osvm:~$
```

- d. To display information about the mounted “/dev/md0”, this command is used,

```
df -h /dev/md0
```

```
jimmyding@osvm:~$ df -h /dev/md0
Filesystem      Size  Used Avail Use% Mounted on
/dev/md0        85M   24K   78M   1% /mnt/rd5
jimmyding@osvm:~$
```

Before

```
jimmyding@osvm:~$ df -h /dev/md0
Filesystem      Size  Used Avail Use% Mounted on
/dev/md0        85M   28K   78M   1% /mnt/rd5
jimmyding@osvm:~$
```

After

Step 7: Spare Disk

- a. The fourth disk is partitioned the same way as the previous three before adding it as a spare.

```
mdadm --add /dev/md0 /dev/sde1
```

```
jimmyding@osvm:~$ sudo mdadm --add /dev/md0 /dev/sde1
mdadm: added /dev/sde1
jimmyding@osvm:~$
```

- b. Once it has been added, we can verify it by viewing the details of the device

```
mdadm --detail /dev/md0
```

```
jimmyding@osvm:~$ sudo mdadm --detail /dev/md0
/dev/md0:
    Version : 1.2
  Creation Time : Fri Apr  4 11:33:46 2025
    Raid Level : raid5
    Array Size : 96256 (94.00 MiB 98.57 MB)
  Used Dev Size : 48128 (47.00 MiB 49.28 MB)
    Raid Devices : 3
    Total Devices : 4
 Persistence : Superblock is persistent

    Update Time : Fri Apr  4 12:03:33 2025
      State : clean
  Active Devices : 3
 Working Devices : 4
 Failed Devices : 0
 Spare Devices : 1


    Layout : left-symmetric
    Chunk Size : 512K

Consistency Policy : resync

           Name : osvm:0 (local to host osvm)
          UUID : 0c7ad10e:637cb255:768be582:50671067
         Events : 19

   Number   Major   Minor   RaidDevice State   /dev/sdb1
     0         8       17         0   active sync
     1         8       33         1   active sync
     3         8       49         2   active sync
     4         8       65         -   spare   /dev/sde1
jimmyding@osvm:~$
```

- The state of the fourth disk is marked as spare.

Step 8: Failed Disk

- a. To mark a disk as failed, the following command is used,

```
mdadm --fail /dev/md0 /dev/<disk name>
```

- b. In my case, the second active disk name is sdc so the command would be

```
mdadm --fail /dev/md0 /dev/sdc1
```

- c. When a disk is marked as “failed”, a spare disk will automatically replace it.

```
mdadm --detail /dev/md0
```

```
jimmyding@osvm:~$ sudo mdadm --detail /dev/md0
/dev/md0:
    Version : 1.2
  Creation Time : Fri Apr  4 11:33:46 2025
    Raid Level : raid5
    Array Size : 96256 (94.00 MiB 98.57 MB)
  Used Dev Size : 48128 (47.00 MiB 49.28 MB)
    Raid Devices : 3
    Total Devices : 4
 Persistence : Superblock is persistent

    Update Time : Fri Apr  4 12:22:24 2025
      State : clean
 Active Devices : 3
Working Devices : 3
 Failed Devices : 1
  Spare Devices : 0


    Layout : left-symmetric
   Chunk Size : 512K

Consistency Policy : resync

       Name : osvm:0 (local to host osvm)
      UUID : 0c7ad10e:637cb255:768be582:50671067
     Events : 38

   Number   Major   Minor   RaidDevice State
    0         8       17         0     active sync   /dev/sdb1
    4         8       65         1     active sync   /dev/sde1
    3         8       49         2     active sync   /dev/sdd1
    1         8       33         -     faulty    /dev/sdc1

jimmyding@osvm:~$
```

Step 9: Removing Failed Disk from Disk Array

- a. To remove a failed disk, the command used,

```
mdadm --remove /dev/md0 /dev/<disk name>
```

- b. In my case it is,

```
mdadm --remove /dev/md0 /dev/sdc1
```

```
mdadm: hot removed /dev/sdc1 from /dev/md0  
jimmyding@osvm:~$
```

- c. Once the failed disk has been removed, the details of /dev/md0 are shown again for verification.

```
mdadm --remove /dev/md0 /dev/sdc1
```

```
jimmyding@osvm:~$ sudo mdadm --detail /dev/md0  
/dev/md0:  
    Version : 1.2  
    Creation Time : Fri Apr  4 11:33:46 2025  
    Raid Level : raid5  
    Array Size : 96256 (94.00 MiB 98.57 MB)  
    Used Dev Size : 48128 (47.00 MiB 49.28 MB)  
    Raid Devices : 3  
    Total Devices : 3  
    Persistence : Superblock is persistent  
  
    Update Time : Fri Apr  4 12:42:06 2025  
    State : clean  
    Active Devices : 3  
    Working Devices : 3  
    Failed Devices : 0  
    Spare Devices : 0  
  
    Layout : left-symmetric  
    Chunk Size : 512K  
  
Consistency Policy : resync  
  
    Name : osvm:0 (local to host osvm)  
    UUID : 0c7ad10e:637cb255:768be582:50671067  
    Events : 39  
  
    Number Major Minor RaidDevice State  
    0        8      17        0     active sync  /dev/sdb1  
    4        8      65        1     active sync  /dev/sde1  
    3        8      49        2     active sync  /dev/sdd1  
jimmyding@osvm:~$
```


Step 10: Unmount Disk Array and Stop It

- a. To unmount the disk array,

```
umount /mnt/rd5
```

- b. To stop the disk array,

```
mdadm --stop /dev/md0
```

```
jimmyding@osvm:~$ sudo mdadm --stop /dev/md0
mdadm: stopped /dev/md0
jimmyding@osvm:~$
```

Task 4: Synchronisation with semaphores

Content of warehouse_solution.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>
#include <string.h>
#include <errno.h>

// Simulation configuration constants
#define MIN_TRAILERS_ARRIVE_TIME 3
#define MAX_TRAILERS_ARRIVE_TIME 4
#define MIN_TRAILERS 3
#define MAX_TRAILERS 10
#define MIN_UNLOAD_TIME 2
#define MAX_UNLOAD_TIME 4
#define MIN_SECURITY_CHECK_TIME 2
#define MAX_SECURITY_CHECK_TIME 3
#define FORKLIFT_OPERATION_TIME 3
#define MAX_FORKLIFTS 2
#define LOADING_BAY_CAPACITY 2
#define TRAILERS_CARRY_CONTAINERS 2

// Shared resource tracking structure
typedef struct {
    // Trailer identifier
    int id;
    // Number of containers to transfer
    int containers_to_unload;
} Trailer;

// Synchronization primitives
pthread_mutex_t security_check_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t security_check_cond = PTHREAD_COND_INITIALIZER;
// Tracks last processed trailer by security
int current_approved_trailer_id = 0;
// Count of trailers awaiting security processing
int remaining_trailers_to_check = 0;

// Controls access to loading bay slots
sem_t loading_bay_sem;
// Global container inventory
int available_containers_count = 0;
pthread_mutex_t containers_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t containers_cond = PTHREAD_COND_INITIALIZER;

// Tracker for security check readiness
int trailer_readiness_status[MAX_TRAILERS] = {0};
pthread_cond_t trailer_readiness_cond = PTHREAD_COND_INITIALIZER;

// Simulation control flags
// Indicates ongoing trailer processing
int simulation_active = 0;

// Function prototypes
void* trailer_processing(void* trailer_data);
void* security_inspection(void* unused_arg);
void* forklift_operation(void* forklift_id);
```

```

int main(int argc, char* argv[]) {
    // Validate input parameters
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <number_of_trailers (3-10)>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int trailer_count = atoi(argv[1]);
    if (trailer_count < MIN_TRAILERS || trailer_count > MAX_TRAILERS) {
        fprintf(stderr, "Number of trailers must be between %d and %d\n",
MIN_TRAILERS, MAX_TRAILERS);
        exit(EXIT_FAILURE);
    }
    printf("Total number of trailers: %d\n", trailer_count);
    remaining_trailers_to_check = trailer_count;

    // Initialize synchronization objects
    sem_init(&loading_bay_sem, 0, LOADING_BAY_CAPACITY);

    // Create security inspection thread
    pthread_t security_thread;
    pthread_create(&security_thread, NULL, security_inspection, NULL);

    // Initialize forklift threads
    pthread_t forklift_threads[MAX_FORKLIFTS];
    for (int i = 0; i < MAX_FORKLIFTS; i++) {
        pthread_create(&forklift_threads[i], NULL, forklift_operation,
(void*)(intptr_t)(i + 1));
    }

    // Create trailer threads with random arrival times
    pthread_t trailer_threads[MAX_TRAILERS];
    Trailer trailers[MAX_TRAILERS];
    for (int i = 0; i < trailer_count; i++) {
        trailers[i].id = i + 1;
        // Each trailer carries 2 containers
        trailers[i].containers_to_unload = TRAILERS_CARRY_CONTAINERS;
        pthread_create(&trailer_threads[i], NULL, trailer_processing,
&trailers[i]);
        // Simulate random arrival times
        sleep(rand() % (MIN_TRAILERS_ARRIVE_TIME - MIN_TRAILERS_ARRIVE_TIME +
1) + MIN_TRAILERS_ARRIVE_TIME);
    }

    // Wait for all trailers to complete processing
    for (int i = 0; i < trailer_count; i++) {
        pthread_join(trailer_threads[i], NULL);
    }
    printf("All trailers joined\n");

    // Signal forklifts to terminate
    pthread_mutex_lock(&containers_mutex);
    simulation_active = 1;
    pthread_cond_broadcast(&containers_cond);
    pthread_mutex_unlock(&containers_mutex);

    // Clean up worker threads
    for (int i = 0; i < MAX_FORKLIFTS; i++) {
        pthread_join(forklift_threads[i], NULL);
    }
    printf("All forklifts joined\n");

    // Terminate security thread and clean up resources
    pthread_cancel(security_thread);
}

```

```

    sem_destroy(&loading_bay_sem);
    pthread_mutex_destroy(&security_check_mutex);
    pthread_cond_destroy(&security_check_cond);
    pthread_mutex_destroy(&containers_mutex);
    pthread_cond_destroy(&containers_cond);

    printf("Program terminates.\n");
    return 0;
}

void* trailer_processing(void* trailer_data) {
    Trailer* trailer = (Trailer*)trailer_data;

    pthread_mutex_lock(&security_check_mutex);
    printf("Trailer-%d: Arrived.\n", trailer->id);
    printf("Trailer-%d: Under checking...\n", trailer->id);

    // Mark trailer as ready for inspection
    trailer_readiness_status[trailer->id - 1] = 1;
    pthread_cond_broadcast(&trailer_readiness_cond);

    // Wait for security clearance
    while (current_approved_trailer_id < trailer->id) {
        pthread_cond_wait(&security_check_cond, &security_check_mutex);
    }
    pthread_mutex_unlock(&security_check_mutex);

    // Access loading bay
    printf("Trailer-%d: Waiting for loading bay...\n", trailer->id);
    sem_wait(&loading_bay_sem);

    // Simulate unloading process
    int processing_time = rand() % (MAX_UNLOAD_TIME - MIN_UNLOAD_TIME + 1) +
MIN_UNLOAD_TIME;
    sleep(processing_time);
    printf("Trailer-%d: Unloaded. Leaving...\n", trailer->id);
    sem_post(&loading_bay_sem);

    // Update global container inventory
    pthread_mutex_lock(&containers_mutex);
    available_containers_count += trailer->containers_to_unload;
    printf("Trailer-%d: Current Total Containers = %d\n",
        trailer->id, available_containers_count);
    pthread_cond_broadcast(&containers_cond);
    pthread_mutex_unlock(&containers_mutex);

    return NULL;
}

void* security_inspection(void* unused_arg) {
    // Explicitly mark unused parameter
    (void)unused_arg;
    int next_trailer_id = 1;

    pthread_mutex_lock(&security_check_mutex);
    printf("Security: Standby\n");

    while (1) {
        if (remaining_trailers_to_check <= 0) {
            pthread_mutex_unlock(&security_check_mutex);
            break;
        }

        // Wait for next trailer to be inspection-ready

```

```

        while (!trailer_readiness_status[next_trailer_id - 1]) {
            pthread_cond_wait(&trailer_readiness_cond, &security_check_mutex);
        }

        printf("Security: Checking trailer-%d\n", next_trailer_id);
        int inspection_duration = rand() % (MAX_SECURITY_CHECK_TIME -
MIN_SECURITY_CHECK_TIME + 1)
            + MIN_SECURITY_CHECK_TIME;
        sleep(inspection_duration);
        printf("Security: Checked & Released-%d\n", next_trailer_id);

        current_approved_trailer_id = next_trailer_id;
        next_trailer_id++;
        remaining_trailers_to_check--;

        pthread_cond_broadcast(&security_check_cond);
        printf("Security: Standby\n");
        pthread_mutex_unlock(&security_check_mutex);
    }

    printf("Security Joined\n");
    return NULL;
}

void* forklift_operation(void* forklift_id) {
    int f_id = (intptr_t)forklift_id;
    struct timespec timeout;

    while (1) {
        pthread_mutex_lock(&containers_mutex);

        // Initialize timeout structure for 10-second wait
        clock_gettime(CLOCK_REALTIME, &timeout);
        // Set 10-second timeout
        timeout.tv_sec += 10;

        // Wait for containers with timeout
        int wait_result = 0;
        while (available_containers_count == 0 && !simulation_active &&
wait_result != ETIMEDOUT) {
            wait_result =
pthread_cond_timedwait(&containers_cond,&containers_mutex,&timeout);
            if (wait_result == ETIMEDOUT) {
                printf("Forklift-%d: Idle for 10 seconds. Shutting down\n",
f_id);
                pthread_mutex_unlock(&containers_mutex);
                return NULL;
            }
        }

        // Check for termination condition
        if (available_containers_count == 0 && simulation_active) {
            pthread_mutex_unlock(&containers_mutex);
            printf("Forklift-%d: Time out. Exit\n", f_id);
            return NULL;
        }

        // Process container
        available_containers_count--;
        printf("Forklift-%d: Moving container (Remaining: %d)\n",f_id,
available_containers_count);
        pthread_mutex_unlock(&containers_mutex);
    }
}

```

```

        // Simulate container transportation
        sleep(FORKLIFT_OPERATION_TIME);
    }

    return NULL;
}

```

Program Output

- a. When the number of trailers is 3,

```

lowjiahao@osvm:~$ ./warehouse_solution 3
Total number of trailers: 3
Security: Standby
Trailer-1: Arrived.
Trailer-1: Under checking...
Security: Checking trailer-1
Security: Checked & Released-1
Security: Standby
Trailer-1: Waiting for loading bay...
Trailer-2: Arrived.
Trailer-2: Under checking...
Security: Checking trailer-2
Trailer-1: Unloaded. Leaving...
Trailer-1: Current Total Containers = 2
Forklift-1: Moving container (Remaining: 1)
Forklift-2: Moving container (Remaining: 0)
Security: Checked & Released-2
Security: Standby
Trailer-3: Arrived.
Trailer-3: Under checking...
Security: Checking trailer-3
Security: Checked & Released-3
Security: Standby
Security Joined
Trailer-2: Waiting for loading bay...
Trailer-3: Waiting for loading bay...
Trailer-2: Unloaded. Leaving...
Trailer-3: Unloaded. Leaving...
Trailer-3: Current Total Containers = 2
Trailer-2: Current Total Containers = 4
Forklift-1: Moving container (Remaining: 3)
Forklift-2: Moving container (Remaining: 2)
Trailer-2: Current Total Containers = 4
Forklift-1: Moving container (Remaining: 3)
Forklift-2: Moving container (Remaining: 2)
All trailers joined
Forklift-1: Moving container (Remaining: 1)
Forklift-2: Moving container (Remaining: 0)
Forklift-1: Time out. Exit
Forklift-2: Time out. Exit
All forklifts joined
Program terminates.
lowjiahao@osvm:~$ █

```